容器化部署實務

從虛擬化到 Kubernetes 的現代應用架構

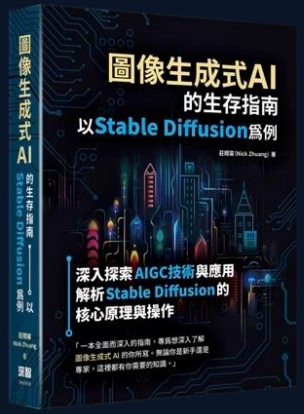
從基礎原理到實戰部署，掌握現代雲原生技術

虛擬化技術 | Docker | Kubernetes | CI/CD 實戰 | AI 容器

Nick Zhuang

承羽智慧 CTO | AI 架構師

- **學歷** 中央大學資工所畢
- **經歷** 曾任上市公司部門技術負責人，現任承羽智慧技術長
- **NVIDIA DLI 講師** 與 NVIDIA 共同發表公開課（A100 膠囊網路 / ConvNeXt + TAO）
- **著作** 「圖像生成式 AI 的生存指南」，繁體中文首本 Stable Diffusion 理論與實務书籍
- **獎項** 113 年度 AIGO 潛力新星盃特優獎
- **認證** Google Cloud 、NVIDIA 多項 AI 認證
- **專長** LLM / RAG / 生成式 AI 、Kubernetes、GCP/AWS 、Cloud DevOps



課程大綱

150 分鐘 · 6 大模組 · 完整實戰流程

01 虛擬化與容器的誕生

15 min

02 Docker 容器技術

20 min

03 Docker Image 與應用打包

15 min

04 Kubernetes 容器編排

45 min

05 容器部署實戰流程

20 min

06 未來架構與 AI 容器

10 min

開場思考題

為什麼 Docker 在 2020 年突然爆紅？

提示：不是 Docker 發明了容器技術...

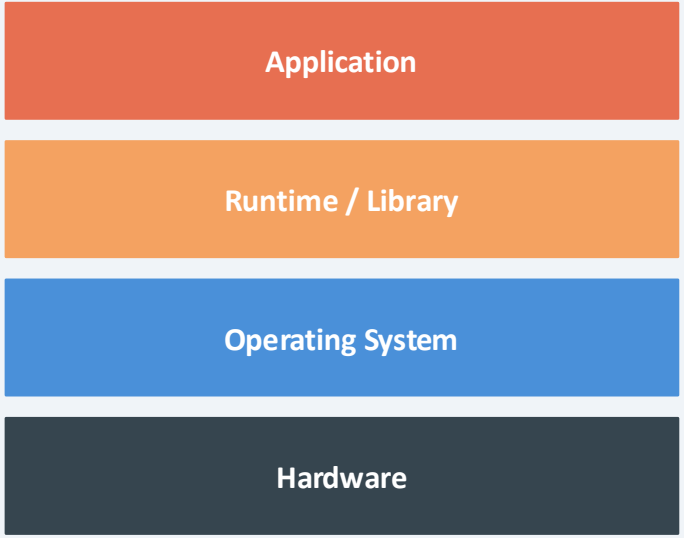
揭曉： Docker 讓 container 變得超簡單！

模組 01

虛擬化與容器的誕生

15 分鐘 | 從傳統部署問題 → 虛擬化 → 容器

傳統部署的痛點



← *早期部署架構（每台機器都要重複安裝）*

❌ 環境不一致

"在我電腦可以跑" 問題層出不窮

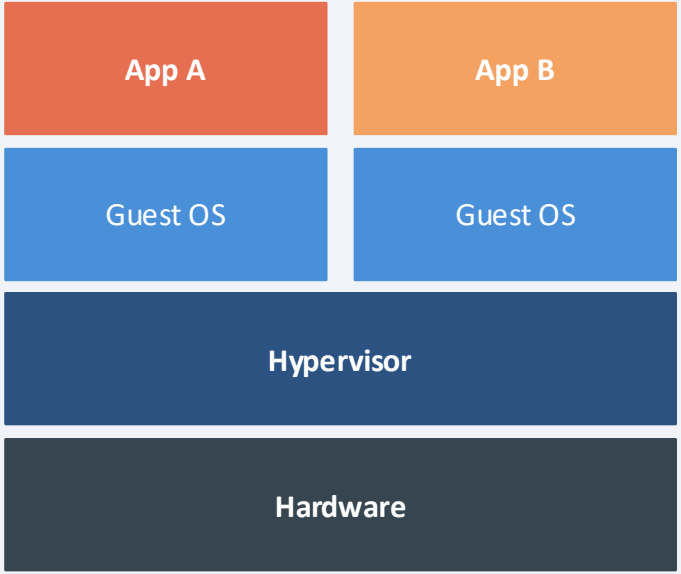
⚡ Server 利用率低

一台 Server 只跑一個應用，資源浪費

🔧 部署複雜

手動安裝依賴、設定環境耗時費力

虛擬化技術：解決之道？



VM 架構

Type 1 (Bare Metal)

例：VMware ESXi, Xen, KVM

直接跑在硬體上，效能好

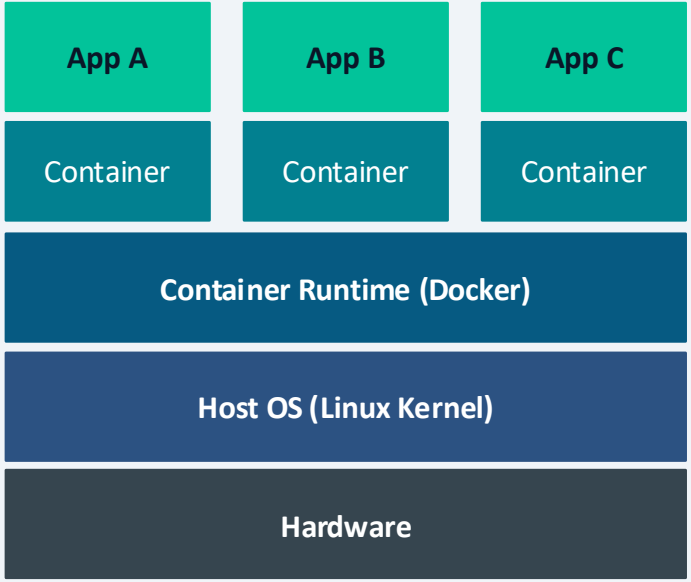
Type 2 (Hosted)

例：VirtualBox, VMware Workstation

跑在 OS 上，適合開發測試

⚠ **VM 的問題：每個 VM 要跑完整 OS → 重、慢、佔資源**

容器：輕量化的革命



容器架構（共享 OS Kernel）

Namespaces

隔離：PID、網路、文件系統

cgroups

控制 CPU、記憶體、I/O 資源

Union FS

Layer 化的文件系統

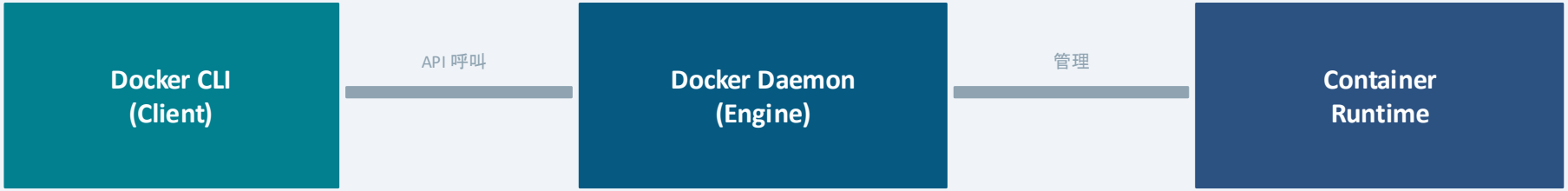
容器 = OS Level Virtualization（共享 Kernel，不需要 Guest OS）

模組 02

Docker 容器技術

20 分鐘 | 核心架構 · 基本指令 · 實際 Demo

Docker 架構概覽



Image

應用程式模板（唯讀）

像是class

Container

執行中的 Image 實例

像是instance

Registry

Image 的儲存庫

Docker Hub、Harbor

Volume

持久化資料存儲

跨Container 共享

📋 常用 Docker 指令

執行容器

```
$ docker run nginx
$ docker run -d -p 8080:80 nginx
$ docker run -it ubuntu bash
```

查看狀態

```
$ docker ps
$ docker ps -a
$ docker logs <container_id>
```

操作容器

```
$ docker exec -it <id> bash
$ docker stop <id>
$ docker rm <id>
```

Image 管理

```
$ docker images
$ docker pull python:3.10
$ docker rmi <image>
```

🎯 DEMO：docker run -p 8080:80 --name nginx nginx → 瀏覽器打開看到 nginx 首頁！

docker inspect 深入解析

```
# 查看容器詳細資訊
$ docker inspect <container_id>

# 取得 IP 位址
$ docker inspect -f '{{.NetworkSettings.Networks.bridge.IPAddress}}' nginx

# 取得掛載的 Volume
$ docker inspect -f '{{json .Mounts}}' nginx | jq .

# 取得環境變數
$ docker inspect -f '{{json .Config.Env}}' nginx | jq .

# 查看 Image 的 Layer 資訊
$ docker image inspect --format='{{json .RootFS.Layers}}' nginx

# 取得容器 PID（可用於 nsenter 進入 namespace）
$ docker inspect -f '{{.State.Pid}}' nginx
```

💡 **-f（--format）支援 Go template 語法，可精確取出任何欄位**

docker logs 日誌管理

查看容器標準輸出（stdout/stderr），是除錯的第一步

```
#查看所有log
$docker logs <container_id>

#即時追蹤log（類似tail -f）
$docker logs -f nginx

#只看最後50行
$docker logs --tail 50 nginx

#顯示時間戳記
$docker logs -t nginx

#只看特定時間之後的log
$docker logs --since 2024-01-01T00:00:00 nginx
$docker logs --since 1h nginx #最近1小時

#顯示錯誤訊息
$docker logs nginx 2>&1 | grep ERROR
```

💡 **Log Driver 可設定為 json-file、journald、fluentd 等**

docker stats 資源監控

```
# 即時監控所有容器（類似 top）
$ docker stats

# CONTAINER    CPU%    MEM USAGE/LIMIT    MEM%    NET I/O
# nginx        0.1%    5.3MiB / 15.5GiB   0.03%   1kB/2kB
# myapp        2.4%    128MiB / 15.5GiB   0.82%   50kB/30kB

# 只監控特定容器
$ docker stats nginx

# 不自動重新整理，只輸出一次（適合 script）
$ docker stats --no-stream

# 自訂格式輸出
$ docker stats --format '{{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}'

# 設定容器資源限制
$ docker run -d --memory='512m' --cpus='1.5' nginx
$ docker update --memory='512m' --cpus='1.5' nginx
```

💡 搭配 **cAdvisor + Prometheus** 可建立完整的容器監控系統

GPU 容器簡介

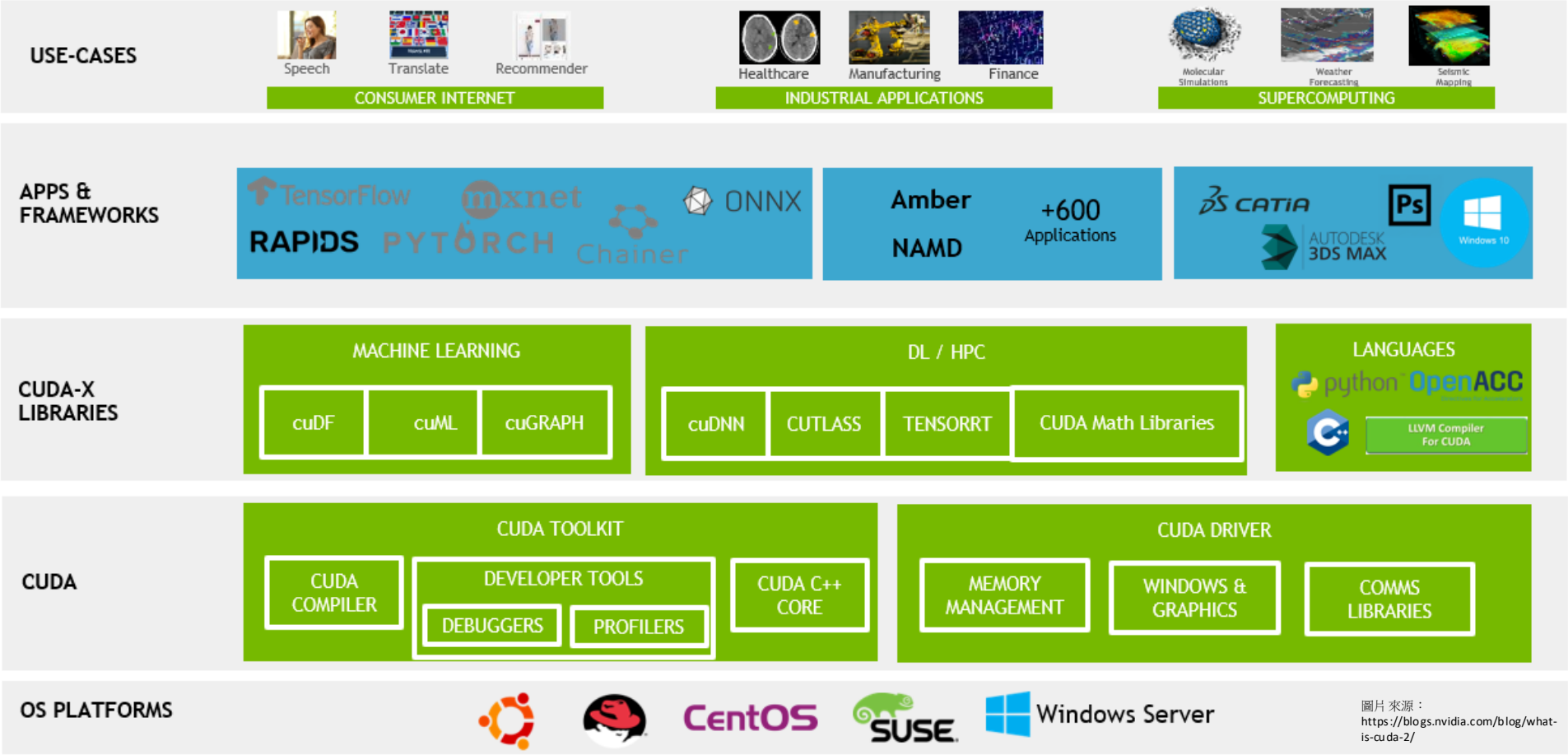
為什麼需要 GPU 容器？

- AI / ML 訓練需要大量浮點運算，GPU 比 CPU 快
- 容器化 GPU 環境確保可重現性（版本一致）
- 多個容器可分享同一張 GPU（MIG / Time-sharing）
- 快速部署 CUDA 環境，不需手動安裝驅動
- Kubernetes 可以調度 GPU 資源到多個節點

GPU 容器技術棧

- NVIDIA Driver：宿主機安裝，容器共享
- CUDA：GPU 計算框架，版本需與 Driver 相符
- cuDNN：深度學習加速函式庫
- NVIDIA Container Toolkit：讓 Docker 存取 GPU
- nvidia-docker2：舊版工具，已被 Toolkit 取代
- NVIDIA Device Plugin：K8s GPU 資源管理

GPU 容器簡介



圖片來源：
https://blogs.nvidia.com/blog/what-is-cuda-2/

NVIDIA Container Toolkit 安裝

```
# 1. 確認 NVIDIA Driver 已安裝
$ nvidia-smi

# 2. 安裝 NVIDIA Container Toolkit
$ curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey \
  | sudo gpg --dearmor -o /usr/share/keyrings/nvidia-ct.gpg
$ curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.list \
  | sudo tee /etc/apt/sources.list.d/nvidia-ct.list
$ sudo apt update && sudo apt install -y nvidia-container-toolkit

# 3. 設定 Docker runtime
$ sudo nvidia-ctk runtime configure --runtime=docker
$ sudo systemctl restart docker

# 4. 驗證安裝
$ docker run --rm --gpus all nvidia/cuda:12.2.0-base-ubuntu22.04 nvidia-smi
```

💡 **宿主機 Driver 版本需 >= 容器內 CUDA 版本所需的最低 Driver**

運行 GPU 容器實戰

```
# 使用所有 GPU
$ docker run --rm --gpus all nvidia/cuda:12.2.0-base-ubuntu22.04 nvidia-smi

# 使用特定 GPU（device 0）
$ docker run --rm --gpus device=0 nvidia/cuda:12.2.0-base-ubuntu22.04 nvidia-smi

# 使用 2 張 GPU
$ docker run --rm --gpus "'device=0,1'" myapp

# 跑 PyTorch GPU 訓練
$ docker run --rm --gpus all \
  -v $(pwd)/data:/data \
  pytorch/pytorch:2.11.0-cuda12.8-cudnn9-runtime \
  python train.py

# 進入 GPU 容器互動式 Shell
$ docker run -it --gpus all --rm \
  nvcr.io/nvidia/tensorflow:25.02-tf2-py3 bash
```

💡 **nvidia-smi** 在容器內可以看到宿主機所有 **GPU** 資訊

模組 03

Docker Image 與應用打包

30 分鐘 | Layer 概念 · Dockerfile · Build & Run

Image Layer 結構

Application Layer	你的程式碼
Dependencies Layer	pip / npm 套件
Runtime Layer	Python / Node.js
Base Image Layer	Ubuntu / Alpine

⚡ 快速 Build

Cache 機制：沒改過的 Layer 不重建

♻ 可重用

多個 Image 共享相同 Base Layer

📦 輕量

只有 Container Layer 是可寫的

← Docker Image 是由多層唯讀Layer 堆疊而成

Dockerfile：打包你的應用

Dockerfile

```
FROM python:3.10-slim
WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

EXPOSE 5000
CMD ["python","app.py"]
```

📦 **Build Image**

```
docker build -t myapp:v1 .
```

🚀 **Run Container**

```
docker run -p 5000:5000 myapp:v1
```

📤 **Push to Registry**

```
docker tag myapp:v1 hub/myapp:v1
docker push hub/myapp:v1
```

Docker Volume 資料持久化

Volume 類型

- Named Volume：由 Docker 管理，最推薦
  docker run -v mydata:/app/data nginx
- Bind Mount：掛載宿主機路徑，適合開發
  docker run -v $(pwd):/app nginx
- tmpfs：掛載在記憶體，容器停止即消失
  docker run --tmpfs /tmp nginx

Volume 常用指令

- # 建立 named volume
  docker volume create mydata
- # 列出所有 volume
  docker volume ls
- # 查看 volume 詳細資訊
  docker volume inspect mydata
- # 刪除未使用的 volume
  docker volume prune
- # 備份 volume 內容
  docker run --rm -v mydata:/data \
  -v $(pwd):/backup busybox \
  tar cvf /backup/backup.tar /data

Docker Compose 多容器應用

```
# docker-compose.yml
services:
  web:
    image: myapp:v1
    ports: ['5000:5000']
    environment:
      - DATABASE_URL=postgresql://db/mydb
    depends_on: [db]
  db:
    image: postgres:15
    volumes: ['pgdata:/var/lib/postgresql/data']
    environment:
      POSTGRES_DB: mydb
      POSTGRES_PASSWORD: secret
volumes:
  pgdata:

# 啟動所有服務
$ docker compose up -d
$ docker compose logs -f
$ docker compose down
```

💡 **docker compose ps** 查看服務狀態；**scale web=3** 水平擴展

模組 04

Kubernetes 容器編排

45 分鐘 | 架構 · 核心概念 · YAML 實作

為什麼需要 Kubernetes？

Docker 只能管理單一主機

- ❌ 無法跨多台機器管理
- ❌ 沒有自動故障恢復
- ❌ 手動負載平衡
- ❌ 滾動更新困難

→

Kubernetes 解決方案

- ✅ 多節點叢集管理
- ✅ 自動故障恢復 (Self-healing)
- ✅ 內建負載平衡
- ✅ 零停機滾動更新

🌐 大規模服務

數百個 Container 同時運行管理

🔄 持續交付

零停機更新、回滾部署版本

📈 彈性擴縮

根據流量自動增減 Pod 數量

🛡️ 高可用性

Pod 掛掉自動重啟、跨節點調度

Kubernetes 架構

Control Plane

API Server

所有操作的入口

Scheduler

決定 Pod 調度到哪個 Node

Controller Manager

確保集群狀態符合期望

etcd

分散式 key-value 存儲（集群狀態）

Worker Node

kubelet

管理 Node 上的 Pod

kube-proxy

網路代理，管理 Service 規則

Container Runtime

containerd / CRI-O

Pod (x N)

實際執行的容器群組

Kubernetes 核心資源

Pod

最小部署單位，包含一個或多個容器

例：一個 Pod 跑一個 Nginx 容器

Pod 死了不會自動重建

Deployment

管理 Pod 的副本數量與滾動更新

例：replicas: 3 → 永遠保持 3 個 Pod

最常用的資源

Service

提供穩定的網路存取入口

例：ClusterIP / NodePort / LoadBalancer

Pod IP 會變，Service IP 不變

Ingress

HTTP/HTTPS 路由規則

例：api.example.com → backend Service

需要 Ingress Controller

Kubernetes YAML 實作

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    spec:
      containers:
      - name: web
        image: myapp:v1
        ports:
        - containerPort: 5000
```

Apply 部署

```
kubectl apply -f deployment.yaml
```

查看狀態

```
kubectl get pods
kubectl get deployments
```

擴縮容

```
kubectl scale deployment
  web-app --replicas=5
```

🎯 宣告式設定：告訴 K8s「我要什麼」而不是「怎麼做」

kubectl 常用指令總覽

查詢資源

- kubectl get pods -A # 所有 namespace
- kubectl get pods -o wide # 顯示 Node IP
- kubectl get all -n default # 所有資源類型
- kubectl get nodes # 列出所有節點
- kubectl get svc # Service 列表
- kubectl get deploy # Deployment
- kubectl get pv,pvc # Storage
- kubectl top pods # 資源使用量

操作資源

- kubectl apply -f manifest.yaml # 套用設定
- kubectl delete -f manifest.yaml # 刪除資源
- kubectl delete pod <name> # 刪除 Pod
- kubectl exec -it <pod> -- bash # 進入容器
- kubectl cp <pod>:/path ./local # 複製檔案
- kubectl port-forward svc/web 8080:80 # 轉發埠
- kubectl rollout restart deploy/web # 重啟
- kubectl scale deploy web --replicas=5 # 擴縮

kubectl describe pod 詳解

```
$ kubectl describe pod web-app-7d9f8b-xyz

# 關鍵欄位解讀：
Name:       web-app-7d9f8b-xyz
Namespace:  default
Node:       worker-node-1/192.168.1.11
Status:     Running
IP:         10.244.1.5
Containers:
  web:
    Image:   myapp:v1
    Port:    5000/TCP
    State:   Running  Started: 2024-01-15
    Limits:  cpu: 500m  memory: 256Mi
    Requests: cpu: 250m  memory: 128Mi
Conditions:  Ready True, PodScheduled True
Events:
  Normal  Scheduled  pod assigned to worker-node-1
  Normal  Pulled     Image pulled successfully
  Normal  Started    Container started
```

💡 **Events 區塊最重要！Pod 起不來時看這裡的錯誤訊息**

kubectl logs 使用指南

```
# 查看 Pod log
$ kubectl logs <pod-name>

# 即時追蹤 log
$ kubectl logs -f <pod-name>

# 只看最後 100 行
$ kubectl logs --tail=100 <pod-name>

# Pod 有多個 Container 時，指定容器名稱
$ kubectl logs <pod-name> -c <container-name>

# 查看已死亡的前一個容器 log（CrashLoopBackOff 時很有用）
$ kubectl logs <pod-name> --previous

# 查看 Label selector 匹配的所有 Pod log
$ kubectl logs -l app=web --all-containers=true

# 取得特定時間範圍的 log
$ kubectl logs <pod-name> --since=1h
```

💡 **CrashLoopBackOff 時先用 --previous 看上一次死掉前的 log**

kubectl describe node — 查看硬體與 GPU 資訊

```
$ kubectl describe node worker-gpu-1

Capacity:                  # 節點總資源
  cpu:                     32
  memory:                  128Gi
  nvidia.com/gpu:          4      # GPU 總數
  pods:                    110

Allocatable:               # 可分配資源
  cpu:                     31800m
  memory:                  125Gi
  nvidia.com/gpu:          4

Allocated resources:       # 已被 Pod 使用
  nvidia.com/gpu:          2/4    # 用了 2 張，剩 2 張

Labels:
  nvidia.com/gpu.product=Tesla-V100-SXM2-32GB
  nvidia.com/gpu.count=4
  nvidia.com/cuda.version=12.2
```

💡 **Labels 區塊顯示 GPU 型號、數量、CUDA 版本，可用於 nodeSelector**

K8s Namespace 管理

Namespace 是什麼？

- K8s 中的虛擬隔離空間，將資源分組管理
- 預設 Namespace：default、kube-system、kube-public
- 同 Namespace 內資源可直接互訪（svc name）
- 跨 Namespace 需用 FQDN：svc.namespace.svc.cluster.local
- 可設定 ResourceQuota 限制資源使用上限

常用 Namespace 指令

- # 建立 namespace
- kubectl create namespace staging
- # 列出所有 namespace
- kubectl get ns
- # 在特定 namespace 操作
- kubectl get pods -n staging
- # 設定預設 namespace
- kubectl config set-context --current \
- --namespace=staging
- # 設定 ResourceQuota
- kubectl apply -f quota.yaml -n staging

ConfigMap & Secret

```
# ConfigMap：從檔案或字串建立
kubectl create configmap app-config \
  --from-literal=APP_ENV=production \
  --from-file=config.yaml

# Secret：base64 自動編碼
kubectl create secret generic db-secret \
  --from-literal=password=mypassword

# 在 Pod 中使用 ConfigMap（環境變數）
env:
  - name: APP_ENV
    valueFrom:
      configMapKeyRef:
        name: app-config
        key: APP_ENV

# 在 Pod 中使用 Secret（掛載為檔案）
volumes:
  - name: secret-vol
    secret:
      secretName: db-secret
```

💡 生產環境建議用 **Sealed Secrets** 或 **Vault** 管理機密

NVIDIA Device Plugin 介紹

什麼是 Device Plugin？

- K8s Device Plugin Framework：讓第三方資源可以被 K8s 管理
- NVIDIA Device Plugin：讓 K8s 感知並調度 GPU 資源
- 透過 DaemonSet 部署在每個 GPU 節點上
- 向 kubelet 廣播節點上的 GPU 數量（nvidia.com/gpu）
- Pod 透過 resources.limits 申請 GPU

安裝 NVIDIA Device Plugin

- # 前置條件：每個節點已安裝 NVIDIA Driver + Container Toolkit

- # 用 Helm 安裝
- helm repo add nvdp \
- https://nvidia.github.io/k8s-device-plugin
- helm install nvdp nvdp/nvidia-device-plugin \
- --namespace kube-system

- # 驗證 DaemonSet 是否正常
- kubectl get pods -n kube-system | grep nvidia

- # 確認節點有 GPU 資源
- kubectl describe node | grep nvidia.com/gpu

GPU Pod 配置與部署

```yaml
# gpu-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: gpu-job
spec:
  containers:
  - name: cuda-container
    image: nvidia/cuda:12.2.0-base-ubuntu22.04
    command: ['sleep infinity']
    resources:
      limits:
        nvidia.com/gpu: 1       # 申請 1 張 GPU
      requests:
        memory: '4Gi'
        cpu: '2'
  nodeSelector:
    nvidia.com/gpu.product: Tesla-V100-SXM2-32GB
  restartPolicy: Never

$ kubectl apply -f gpu-pod.yaml
$ kubectl logs gpu-job
```

💡 **GPU 只能整數申請（1,2,3...），分數 GPU 需用 MIG 或 Time-Slicing**

實戰：部署 Nginx 網頁服務

```
# 1. 建立 Deployment
kubectl create deployment nginx --image=nginx:latest
# 或套用 YAML
kubectl apply -f nginx-deploy.yaml

# 2. 暴露 Service（NodePort）
kubectl expose deployment nginx --port=80 --type=NodePort

# 3. 確認 Pod 狀態
kubectl get pods -l app=nginx
kubectl get svc nginx

# 4. 取得 NodePort
kubectl get svc nginx -o jsonpath='{.spec.ports[0].nodePort}'
# → 存取：http://<node-ip>:<nodeport>

# 5. 更新 Image（零停機滾動更新）
kubectl set image deployment/nginx nginx=nginx:1.25
kubectl rollout status deployment/nginx

# 6. 回滾
kubectl rollout undo deployment/nginx
```

💡 **kubectl rollout history deployment/nginx** 查看版本歷史

Service 類型詳解

四種 Service 類型

- ClusterIP（預設）：只在叢集內存取
- → 適合微服務之間互訪
- → Cluster IP: 10.96.x.x
- NodePort：暴露在每個 Node 的指定埠
- → 適合測試或非生產環境
- → 埠範圍：30000-32767
- LoadBalancer：雲端廠商提供外部 LB
- → GKE / EKS / AKS 自動建立
- ExternalName：DNS CNAME 轉換
- → 把外部服務對應到 K8s 名稱

Service YAML 範例

```
apiVersion: v1
kind: Service
metadata:
  name: web-svc
spec:
  selector:
    app: web
  type: NodePort
  ports:
    - port: 80
      targetPort: 5000
      nodePort: 30080
# 查看 Endpoints
kubectl get endpoints web-svc
```

LoadBalancer 實戰案例

```
#MetalLB：裸機K8s的LoadBalancer解決方案

#1.安裝MetalLB
kubectl apply -f https://raw.githubusercontent.com/metallb/metallb/v0.14.3/config/manifests/metallb-native.yaml

#2.設定IP池（使用本地IP段）
#metallb-config.yaml
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: local-pool
  namespace: metallb-system
spec:
  addresses: ['192.168.1.200-192.168.1.250']

#3.建立LoadBalancer Service
kubectl expose deploy nginx --type=LoadBalancer --port=80

#4.取得外部IP
kubectl get svc nginx
#NAME   TYPE       EXTERNAL-IP    PORT
#nginx  LoadBalancer  192.168.1.200  80
```

💡 **雲端環境（GKE/EKS/AKS）無需 MetalLB，自動建立雲端 LB**

Ingress Controller 配置

```
# 安裝 NGINX Ingress Controller
helm install ingress-nginx ingress-nginx/ingress-nginx \
  --namespace ingress-nginx --create-namespace

# ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - host: api.example.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: api-svc
            port: {number: 80}
```

💡 搭配 cert-manager 可自動申請 Let's Encrypt TLS 憑證

HPA 水平自動擴縮容

```
#建立HPA（目標CPU使用率50%）
kubectl autoscale deployment web \
  --cpu-percent=50 --min=2 --max=10

#或用YAML定義（更靈活）
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

💡 需安裝 Metrics Server 才能取得 CPU/記憶體數據

K8s 生產環境最佳實踐

資源管理

- 永遠設定 requests 和 limits，避免資源搶佔
- 設定 PodDisruptionBudget 確保滾動更新不中斷服務
- 使用 Liveness / Readiness Probe 讓 K8s 自動偵測健康狀態
- 使用 ResourceQuota 限制各 Namespace 的資源上限
- Pod Anti-Affinity 避免所有副本在同一個 Node

安全與可觀測性

- 使用 RBAC 最小權限原則
- NetworkPolicy 限制 Pod 間的網路存取
- ImagePullPolicy: Always 避免使用過時 Image
- Container SecurityContext：非 root 用戶運行
- Prometheus + Grafana 監控 Pod 指標
- EFK（Elasticsearch + Fluentd + Kibana）集中日誌

K8s 常見問題排查

Pod 無法啟動

- ImagePullBackOff → Image 名稱錯誤或 Registry 未授權
- kubectl describe pod 看 Events，確認 imagePullSecrets
- CrashLoopBackOff → 容器啟動後立刻死亡
- kubectl logs --previous 看上次死亡前的 log
- Pending → 資源不足或 nodeSelector 不符
- kubectl describe pod 看 Events 中的 Warning
- OOMKilled → 記憶體超過 limits，被 K8s 殺掉
- 調高 limits.memory 或找記憶體洩漏

常用排查指令

- # 查看所有 Pod 狀態
- kubectl get pods -A
- # 查看 Node 資源使用
- kubectl top nodes
- # 查看 K8s 系統元件 log
- kubectl logs -n kube-system <pod>
- # 進入問題 Pod 的容器
- kubectl exec -it <pod> -- sh
- # 查看 API Server 事件
- kubectl get events --sort-by=.lastTimestamp
- # 模擬排程結果
- kubectl describe pod | grep -A5 Events

模組 05

容器部署
實戰流程

20 分鐘 | 完整 CI/CD Pipeline · Registry · 實戰 Demo

🚀 完整容器化部署流程

1
Code
開發應用程式

→

2
Dockerfile
定義打包規則

→

3
Build Image
docker build

→

4
Push Registry
Docker Hub / Harbor

→

5
K8s Deploy
kubectl apply

📦 Registry 選擇

- Docker Hub：公開免費，適合個人/開源
- Harbor：自建，適合企業私有環境
- AWS ECR / GCR：雲端廠商，整合 K8s 方便

⚙ CI/CD 工具整合

- GitHub Actions：Push 自動觸發 build & deploy
- GitLab CI：企業常用，功能完整
- ArgoCD：GitOps 風格，K8s 原生

實戰 DEMO

① 寫一個 Flask API ② 建立 Dockerfile ③ docker build & push ④ 部署到 Kubernetes ⑤ kubectl get pods 確認狀態

動手實作是學習最有效的方式

參考步驟

```
#步驟1：移除taint（只需做一次）
kubectl taint nodes <control-plane-hostname> node-role.kubernetes.io/control-plane:NoSchedule-

#步驟2：build + 匯入containerd
cd course-labs/flask-k8s-demo
chmod +x build-and-push.sh deploy.sh
./build-and-push.sh

#步驟3：部署（會自動檢查taint是否已移除）
./deploy.sh

#步驟4：測試負載均衡（多跑幾次看pod_name切換）
NODE_IP=$(kubectl get nodes -o jsonpath='{.items[0].status.addresses[?(@.type=="InternalIP")].address}')
for i in $(seq 1 6); do
  curl -s http://${NODE_IP}:30500/api/info | python3 -m json.tool | grep pod_name
done

#課後清理
kubectl delete -f k8s/
kubectl taint nodes <control-plane-hostname> node-role.kubernetes.io/control-plane:NoSchedule
```

Harbor 私有 Registry 設置

```
# 使用 Helm 安裝 Harbor
helm repo add harbor https://helm.goharbor.io
helm install harbor harbor/harbor \
  --namespace harbor --create-namespace \
  --set expose.type=nodePort \
  --set expose.tls.enabled=false \
  --set externalURL=http://192.168.1.200

# 登入 Harbor
docker login 192.168.1.200 -u admin -p Harbor12345

# Push Image 到 Harbor
docker tag myapp:v1 192.168.1.200/myproject/myapp:v1
docker push 192.168.1.200/myproject/myapp:v1

# 掃描 Image 漏洞（Trivy）
trivy image 192.168.1.200/myproject/myapp:v1
```

💡 **Harbor 支援 Proxy Cache，可快取 Docker Hub Image 加速拉取**

Prometheus + Grafana 監控

Prometheus 監控

- 拉取（Pull）式指標收集，適合 K8s 動態環境
- kube-state-metrics：K8s 資源狀態指標
- node-exporter：節點 CPU/記憶體/磁碟指標
- Pod 加 annotation 即可自動被 Prometheus 抓取
- PromQL 查詢語言，可做複雜的聚合計算
- AlertManager：設定告警規則，發送 Slack/Email

一鍵安裝（kube-prometheus-stack）

- helm repo add prometheus-community \
  https://prometheus-community.github.io/helm-charts
- helm install kube-prom-stack \
  prometheus-community/kube-prometheus-stack \
  --namespace monitoring --create-namespace
- # 查看所有元件
- kubectl get pods -n monitoring
- # 進入 Grafana UI
- kubectl port-forward svc/kube-prom-stack-grafana \
  3000:80 -n monitoring
- # 預設帳密：admin / prom-operator

模組 06

未來架構與 AI 容器化

10 分鐘 | AI Infrastructure · GPU · 展望

AI 時代的容器架構

AI Infrastructure = Container + GPU + Scheduler

AI Workloads 特性

- 需要 GPU 資源
- 大量記憶體需求
- 分散式訓練

NVIDIA Container Toolkit

- 讓容器存取 GPU
- nvidia-docker2
- 標準化 GPU 環境

K8s GPU 排程

- limits.nvidia.com/gpu: 1
- 多 Node GPU 叢集
- 自動化 GPU 分配

AI 框架容器化

- PyTorch / TensorFlow
- Jupyter Notebook
- MLflow 實驗追蹤

GPU Device Plugin 深入解析

Device Plugin 運作原理

- 每個 GPU 節點運行一個 nvidia-device-plugin DaemonSet Pod
- Plugin 透過 gRPC 向 kubelet 的 DevicePlugin API 註冊
- 廣播資源：nvidia.com/gpu: N（節點上的 GPU 數量）
- Pod 申請 GPU 時，kubelet 呼叫 Plugin 的 Allocate() 分配
- Plugin 回傳 CUDA_VISIBLE_DEVICES 環境變數給容器

GPU Sharing 方案

- 整數分配（預設）：每個 Pod 獨占整張 GPU
- MIG（Multi-Instance GPU）：A100 / H100 專屬
  → 硬體層切片，完全隔離，最安全
- GPU Time-Slicing：同一張 GPU 分時給多 Pod
  → 軟體層共享，適合輕量推論任務
- vGPU（NVIDIA vGPU）：VM 層的 GPU 虛擬化
  → 需授權，適合 VM 環境

多 GPU 工作負載設計

```
# 多 GPU Pod（單節點4 GPU）
resources:
 limits:
  nvidia.com/gpu: 4

# 多節點分散式訓練（使用 PyTorch DDP + K8s Job）
apiVersion: batch/v1
kind: Job
metadata:
 name: dist-train
spec:
 parallelism: 2       # 2 個 Pod 並行
 completions: 2
 template:
  spec:
   containers:
   - name: trainer
    image: pytorch/pytorch:2.1.0-cuda12.1-cudnn8-runtime
    command: ['torchrun', '--nproc_per_node=4',
          '--nnodes=2', 'train.py']
    resources:
     limits:
      nvidia.com/gpu: 4
```

💡 更完整的分散式訓練可使用 **Kubeflow Training Operator**

AI 容器最佳實踐

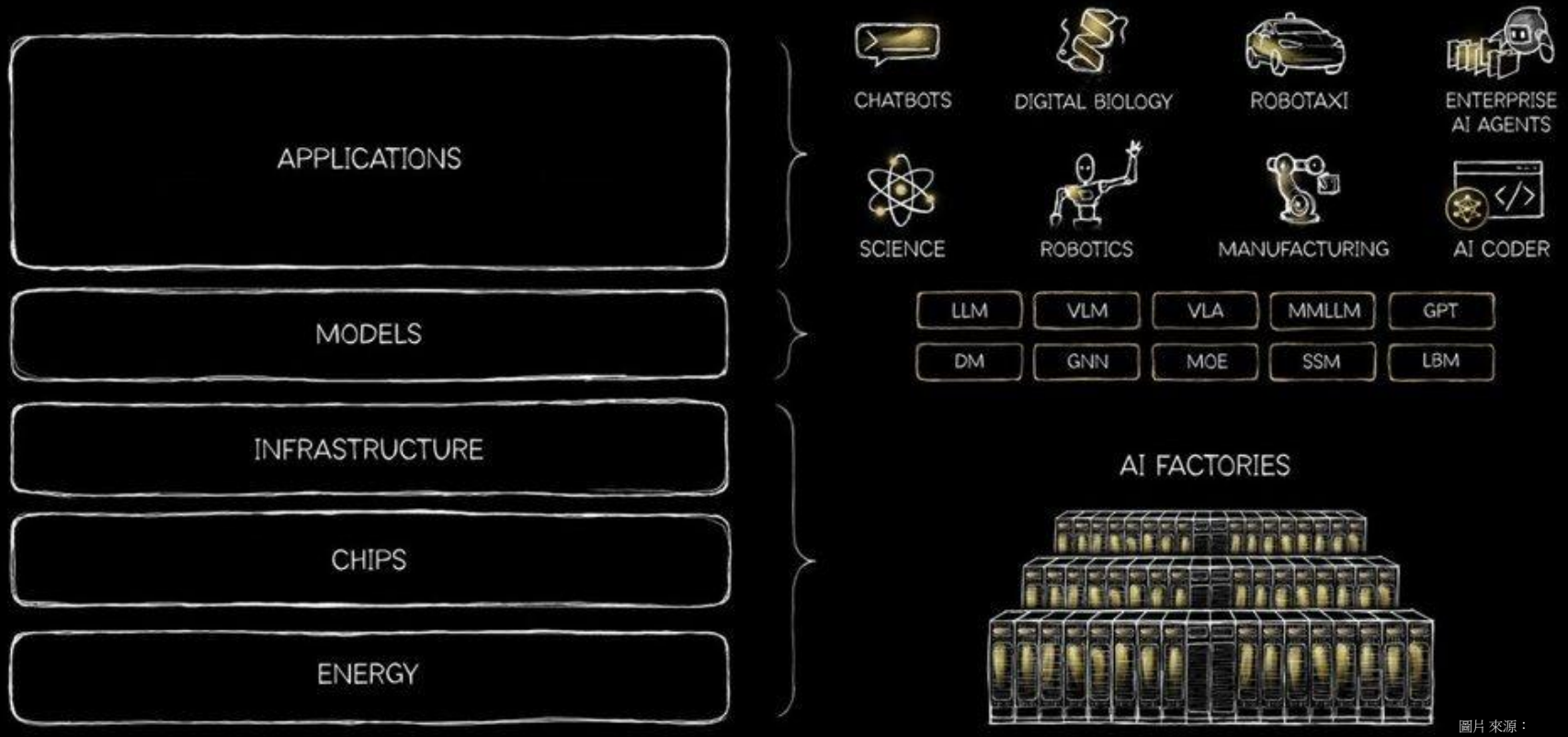
Image 管理

- 使用官方 NVIDIA NGC Image 作為 Base
  nvcr.io/nvidia/pytorch:24.01-py3
- Multi-stage Build 減少最終 Image 大小
- 固定 CUDA + cuDNN + 框架版本，確保可重現
- 模型檔案掛載 PVC，不打進 Image
- 使用 .dockerignore 排除大型資料集

K8s 排程最佳實踐

- 設定 nodeSelector 或 Affinity 指定 GPU 型號
- 訓練任務用 K8s Job；推論服務用 Deployment
- 設定 GPU 資源 limits = requests（確保獨占）
- 推論服務搭配 HPA 依流量自動擴縮
- 使用 PriorityClass 讓關鍵推論任務優先排程
- DCGM Exporter：匯出 GPU 指標到 Prometheus

AI 容器最佳實踐-GTC的啟示



圖片 來源：
https://www.peoplenews.tw/articles/economic-news/21260

Q&A

思考問題

Q1. VM 會被 Container 完全取代嗎？

Q2. Kubernetes 是不是太複雜了？值得學嗎？

Q3. Serverless 會不會取代 Kubernetes？

課程總結

- 🔷 虛擬化解決了硬體利用率，但 VM 太重
- 🐳 Docker 讓容器變簡單：Build → Run → Ship
- 📦 Image 的 Layer 結構讓部署快速且可重現
- ⚛️ Kubernetes 解決大規模容器的編排問題
- 🚀 CI/CD + Container = 現代化應用交付標準
- 🤖 AI 時代：容器 + GPU 是新的基礎設施

謝謝大家 🎉

Q & A

延伸學習

課後補充

Helm · RBAC · Istio · 學習路線圖 · 指令速查表

Helm — Kubernetes 套件管理員

什麼是 Helm？

- Kubernetes 的 apt / yum
- 將 K8s YAML 打包為可重用的 Chart
- 一行指令部署複雜應用
- 支援版本管理與回滾
- 官方 Hub：ArtifactHub.io

常用指令

- helm repo add <name> <url> # 加入 repo
- helm repo update # 更新清單
- helm search repo <keyword> # 搜尋
- helm install <rel> <chart> # 安裝
- helm upgrade <rel> <chart> # 升級
- helm rollback <rel> <rev> # 回滾
- helm uninstall <rel> # 移除
- helm list # 查看已安裝

Kubernetes RBAC 權限控制

核心概念

- Role / ClusterRole：定義可操作的資源與動作
- RoleBinding / ClusterRoleBinding：將 Role 綁定給 User 或 ServiceAccount
- Namespace 隔離：Role 限定在單一 ns
- ClusterRole：跨 Namespace 全局權限
- ServiceAccount：Pod 的身份識別

RBAC YAML 範例

```
kind: Role
rules:
- apiGroups: [""]
  resources: [pods]
  verbs: [get,list]
---
kind: RoleBinding
subjects:
- kind: ServiceAccount
  name: my-sa
roleRef:
  kind: Role
  name: pod-reader
```

GitOps 與 ArgoCD

什麼是 GitOps？

- 以 Git 作為唯一真相來源（Single Source of Truth）
- K8s 狀態完全由 Git repository 定義
- 自動同步：Git 有變更 → 自動部署到 K8s
- 易於審計（PR review）、回滾（git revert）
- 代表工具：ArgoCD、Flux CD

ArgoCD 快速上手

- # 安裝 ArgoCD
- kubectl create ns argocd
- kubectl apply -n argocd -f \
- https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
- # 取得初始密碼
- kubectl -n argocd get secret argocd-initial-admin-secret \
- -o jsonpath='{.data.password}' | base64 -d
- # 登入 UI
- kubectl port-forward svc/argocd-server -n argocd 8080:443

Service Mesh — Istio 微服務治理

Service Mesh 解決什麼？

- 服務間流量加密（mTLS）
- 細粒度流量控制（A/B、Canary）
- 自動重試、熔斷、限流
- 分散式追蹤（Jaeger / Zipkin）
- 統一可觀測性（Kiali Dashboard）
- 應用程式無需修改程式碼

Istio 核心元件

- istiod（控制平面）：Pilot + Citadel + Galley
- Envoy Sidecar：注入到每個 Pod
- VirtualService：路由規則定義
- DestinationRule：負載均衡 / mTLS 策略
- Gateway：南北向流量入口
- PeerAuthentication：強制 mTLS

課程學習路線圖

- 【Module 1】虛擬化與容器 → 從傳統部署到 VM 到容器化的演進
- 【Module 2】Docker 容器基礎 → Dockerfile、Image 打包、docker-compose、GPU 容器
- 【Module 3】Docker 進階 → docker inspect、docker logs、Volume、Registry（Harbor）
- 【Module 4】Kubernetes 核心 → Deployment、Service、Ingress、HPA、GPU Pod
- 【Module 5】K8s 進階 → kubectl 速查、RBAC、Helm、Service Mesh（Istio）
- 【Module 6】CI/CD GitOps → ArgoCD、Harbor、Prometheus + Grafana 監控
- 【Module 7】AI / GPU 部署 → NVIDIA Device Plugin、分散式訓練 Job、最佳實踐
- 推薦認證路線：CKA → CKAD → CKS → NVIDIA DLI GPU Operator

Docker 常用指令速查表

基礎操作

```
docker pull <image>:<tag>
docker images
docker rmi <image>
docker run -it <image> bash
docker run -d -p 8080:80 <image>
docker ps          # 執行中
docker ps -a       # 全部
docker stop/start/restart <id>
docker rm <id>
docker exec -it <id> bash
docker logs -f <id>
docker logs --tail=100 <id>
docker inspect <id>
docker stats
```

建置 / 進階

```
docker build -t myapp:v1 .
docker build --no-cache -t myapp:v1 .
docker tag myapp:v1 registry/myapp:v1
docker push registry/myapp:v1
docker volume ls
docker volume create mydata
docker network ls
docker network inspect bridge
docker compose up -d
docker compose down
docker compose logs -f
docker compose ps
# GPU 容器
docker run --gpus all nvidia/cuda nvidia-smi
```

kubectl 常用指令速查表

查詢 / 除錯

```
kubectl get pods -A
kubectl get pods -o wide
kubectl get nodes
kubectl get svc,deploy,pv,pvc
kubectl describe pod <pod>
kubectl describe node <node>
kubectl logs -f <pod>
kubectl logs --tail=100 <pod>
kubectl logs --previous <pod>
kubectl top pods
kubectl top nodes
kubectl get events --sort-by=.lastTimestamp
kubectl get events --field-selector reason=Failed
kubectl describe node | grep nvidia.com/gpu
```

操作 / 部署

```
kubectl apply -f manifest.yaml
kubectl delete -f manifest.yaml
kubectl delete pod <pod>
kubectl exec -it <pod> -- bash
kubectl port-forward svc/web 8080:80
kubectl scale deploy web --replicas=5
kubectl set image deploy/web web=myapp:v2
kubectl rollout status deploy/web
kubectl rollout history deploy/web
kubectl rollout undo deploy/web
kubectl rollout restart deploy/web
kubectl create namespace staging
kubectl config set-context --current \
  --namespace=staging
```